Informed Search

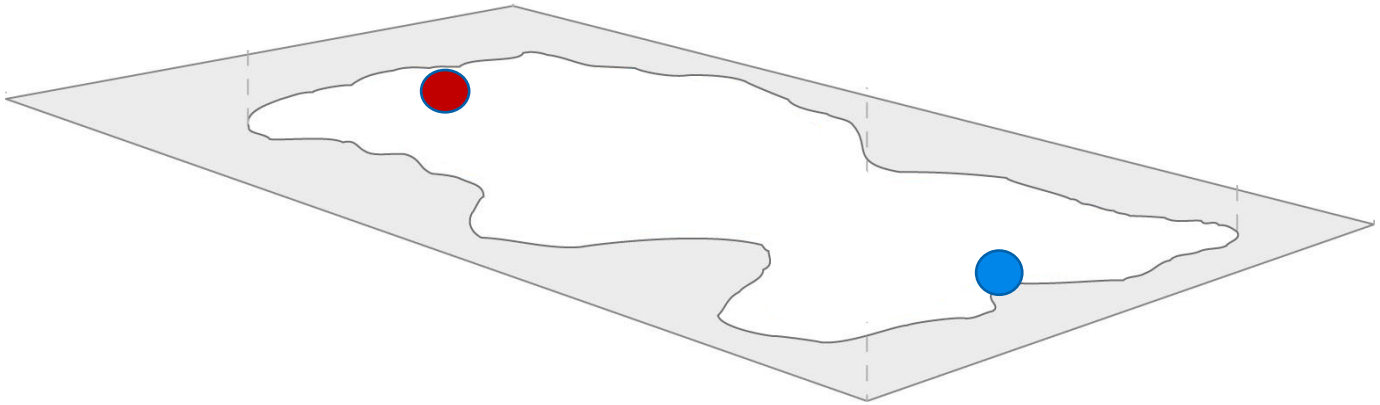
EECS 492

Introduction to Artificial Intelligence

Today's Class

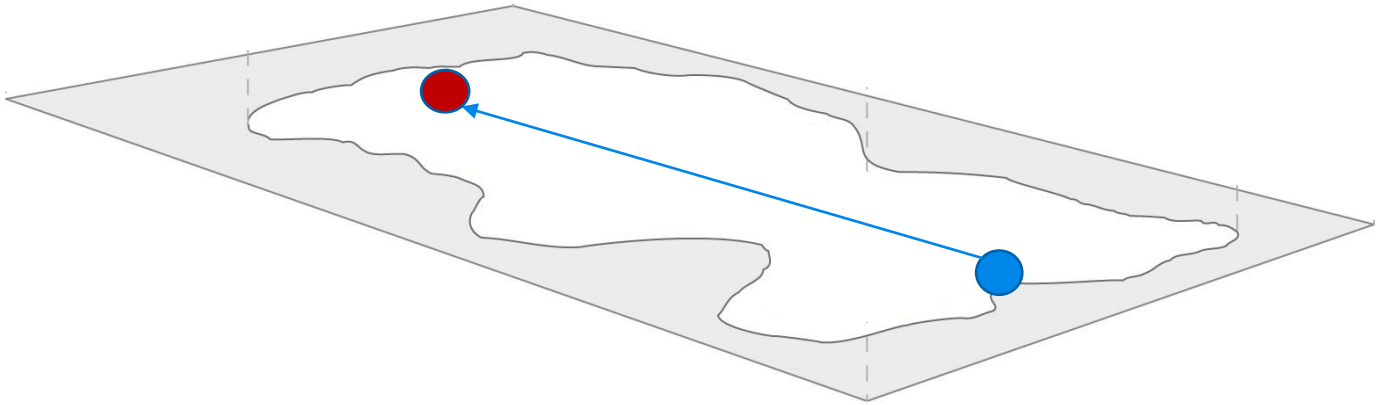
- Informed Search
  - Greedy best-first search
  - A*
  - Heuristics

Information Matters



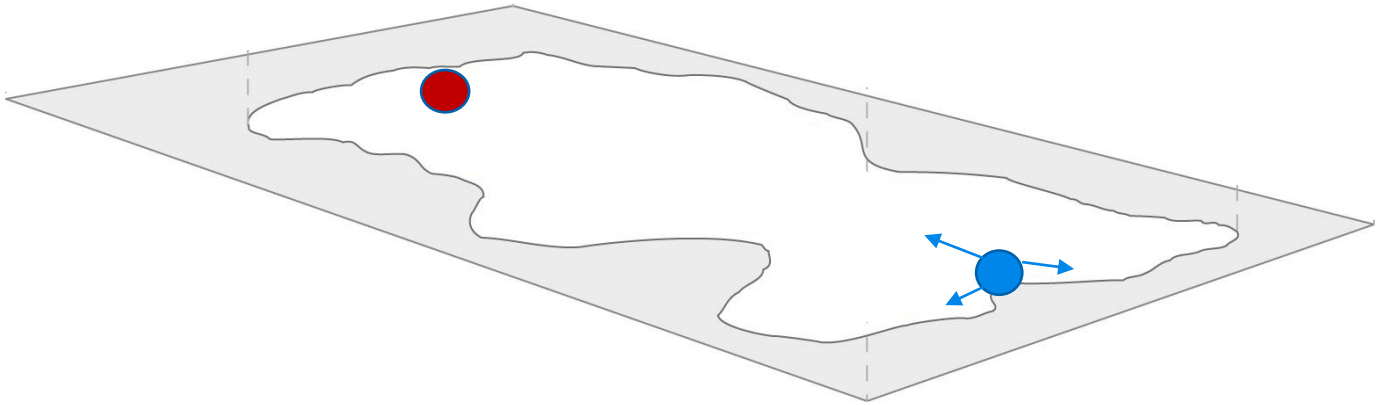
https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Information Matters



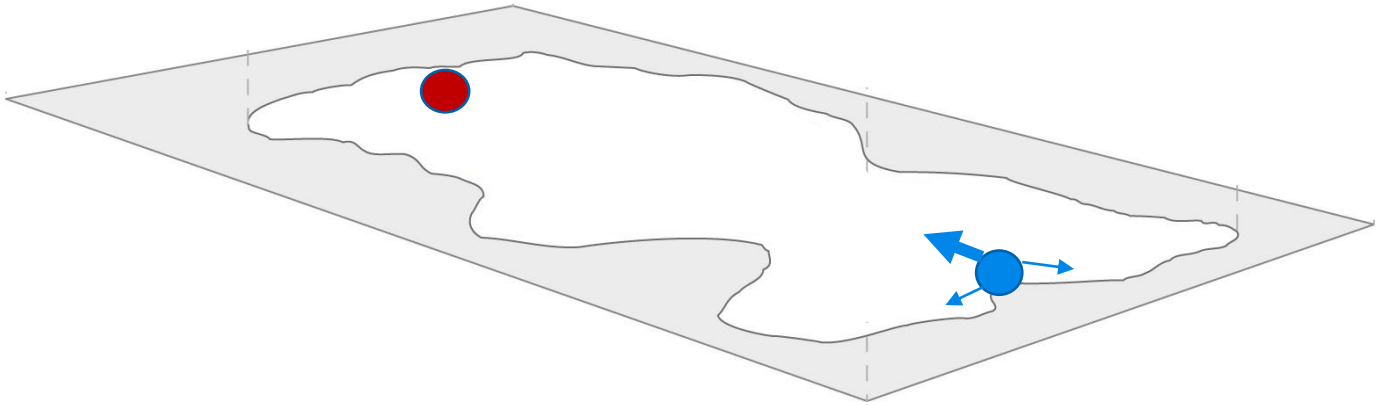
https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Information Matters



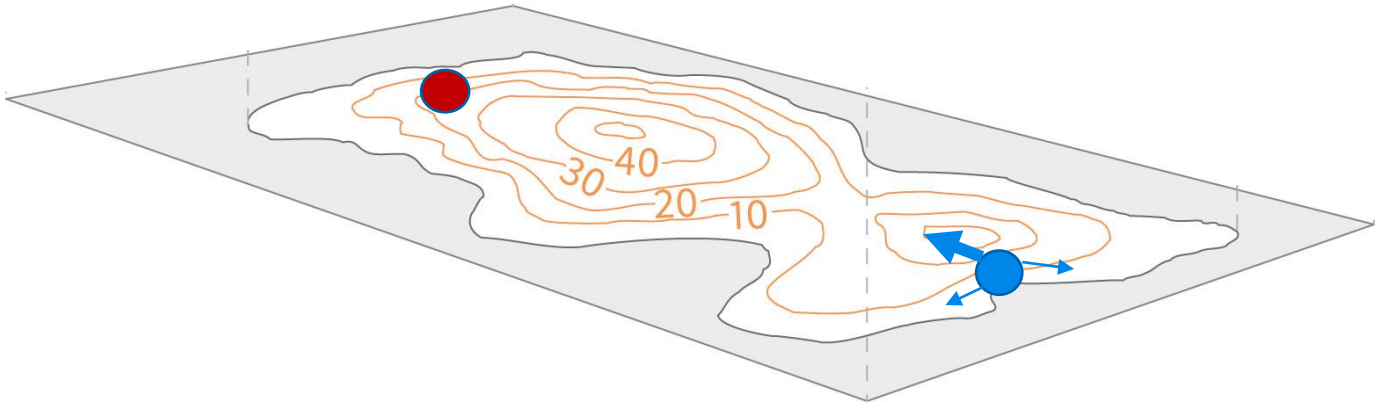
https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Information Matters



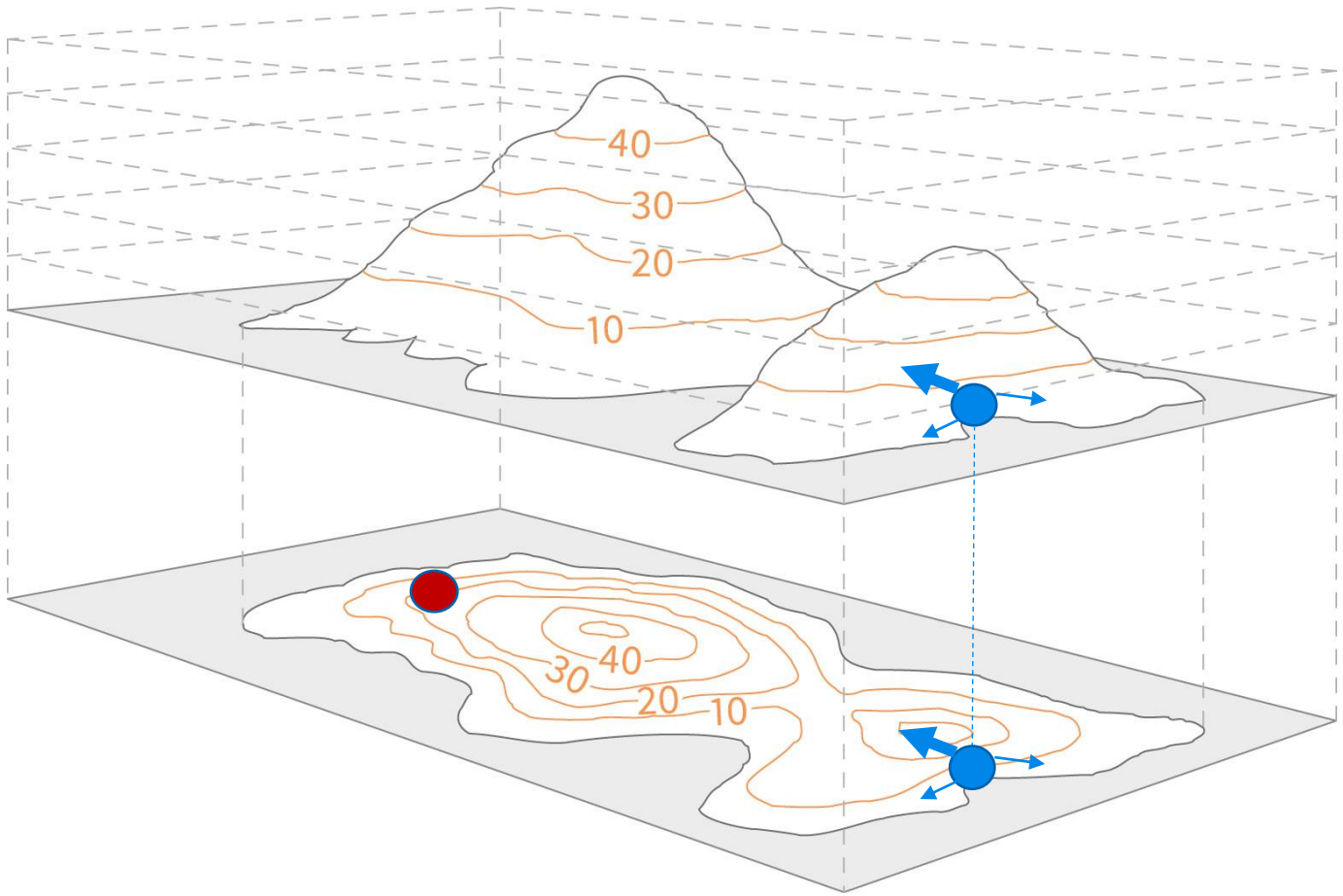
https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Information Matters



https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Information Matters



https://getoutside.ordnancesurvey.co.uk/site/uploads/images/assets/Web%20images/Contours-and-relief.jpg

Informed Search

- The process of using additional information to guide the search strategy.

- The *how* (how we use the information) will define the type of search.

- Evaluation function *f(n)*
  - Expand the node with the best *f(n)*
  - Where does the extra information come in? Part of *f(n)* is *h(n)* – the heuristic function

Heuristic Function

- Written as *h(n)*:
  - Estimated cost of the cheapest path from state at node n to the goal state
  - Depends on the state not on the node (vs. g(n))

- Properties:
  - Arbitrary non-negative values
  - Problem-specific functions
  - Constraint: h(goal node) = 0

Informed Search: Greedy Best-first Search

Greedy Best-First Search

- Goal: expand the node closest to the goal
  - Motivation: this will lead to the goal most quickly (e.g., smallest $h(n)$)
  - Evaluation function: heuristic → $f(n) = h(n)$

- For a map, heuristic function could be the straight-line distance between a state and the goal state

GBFS in the general search architecture

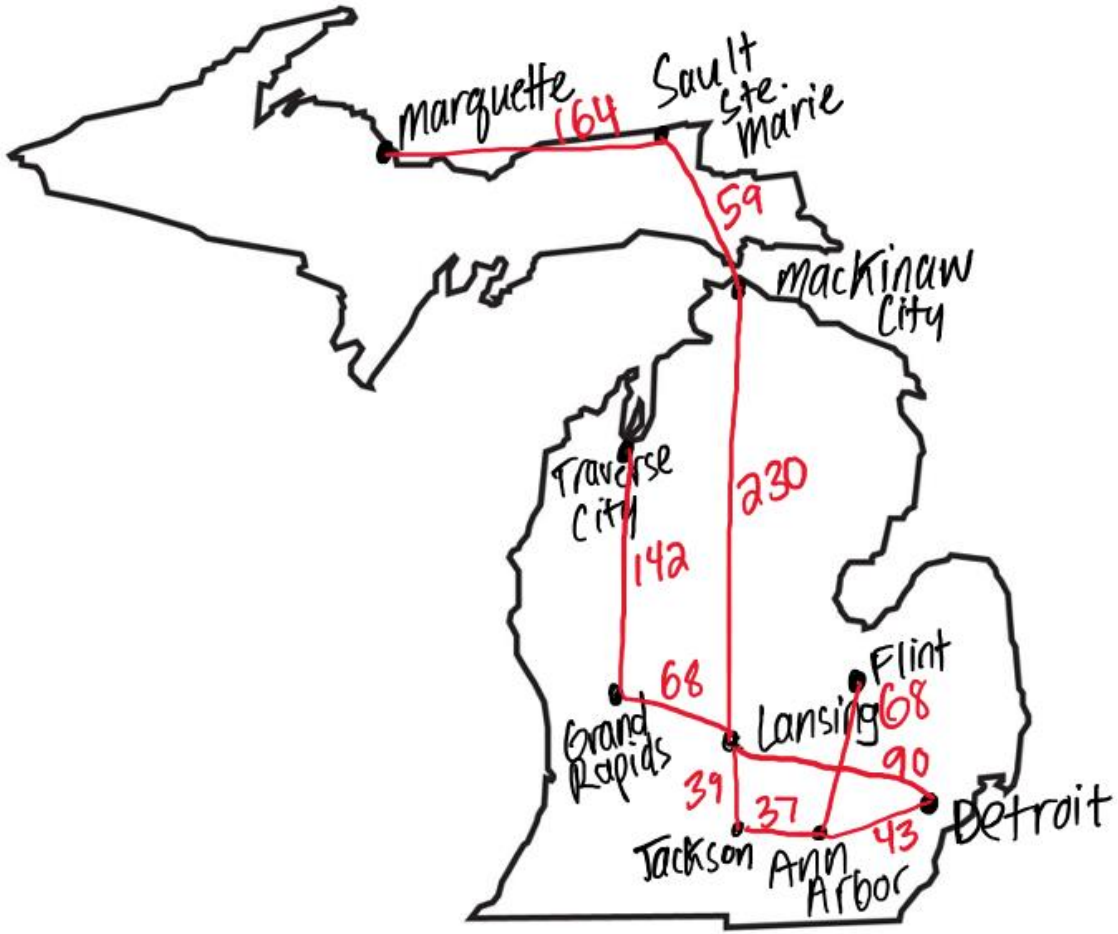
```
function GREEDY-BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Reached: if graph search
Frontier: priority queue (heuristic)

When poll is active respond at **PollEv.com/492**

Exercise: Greedy Best-first-search from Detroit to Flint (graph search)



City	Straight-line distance to Flint (miles)
Ann Arbor	51
Detroit	58
Flint	0
Grand Rapids	100
Jackson	65
Lansing	48
Mackinaw City	198
Marquette	306
Sault Ste. Marie	243
Traverse City	154

Greedy BFS -- Optimality

- Is Greedy BFS optimal?

- Is Greedy BFS complete?
  - We should consider finite spaces and infinite spaces

Informed Search: A* Search

A* Search

- This search method combines:
  - The heuristic function (e.g., straight-line distance)
  - The path-cost (e.g., total distance traveled)
  - $f(n) = g(n) + h(n)$
    - Can be thought of as estimated cost of the cheapest solution through n

- Comparison:
  - Uniform-cost search: $f(n) = g(n)$
  - Greedy best-first-search: $f(n) = h(n)$
  - A*: $f(n) = g(n) + h(n)$

A* in the general search architecture

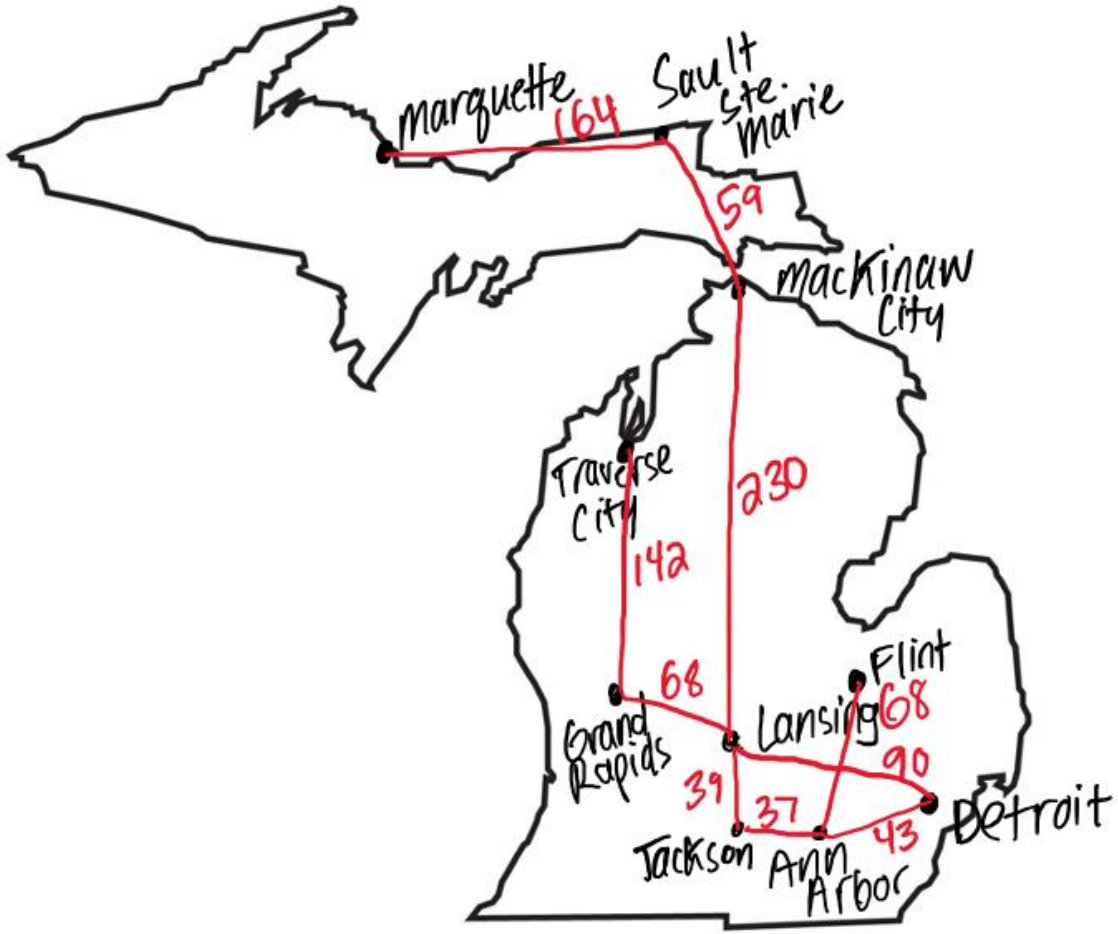
```
function A*-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node

    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Reached: if graph search
Frontier: priority queue (heuristic+pathCost)

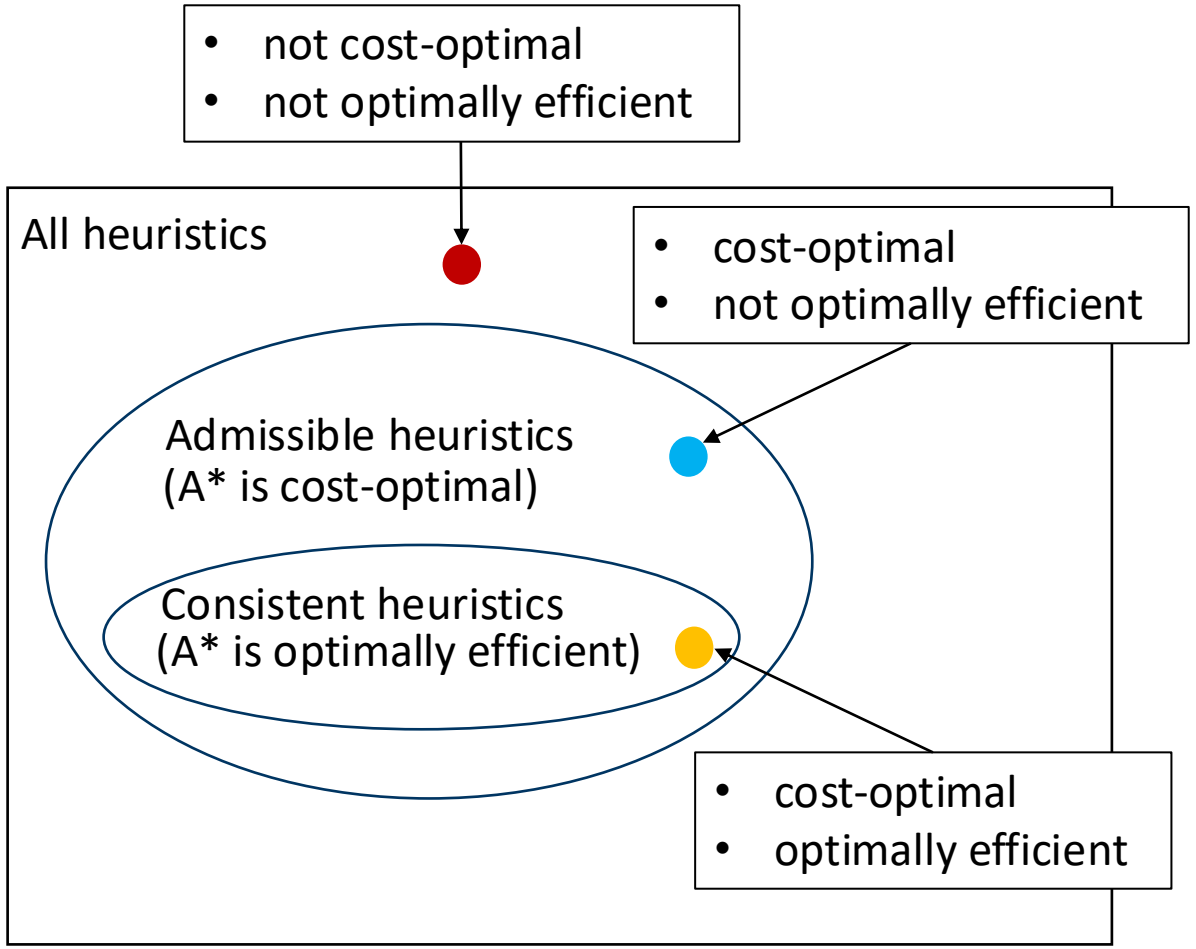
When poll is active respond at **PollEv.com/492**

Exercise: A* from Lansing to Marquette (graph search)



City	Straight-line distance to Marquette (miles)
Ann Arbor	347
Detroit	364
Flint	306
Grand Rapids	260
Jackson	331
Lansing	298
Mackinaw City	147
Sault Ste. Marie	157
Traverse City	152

A* Optimality is Based on Heuristic Characteristics



Optimally efficient:
No algorithm can expand fewer nodes and still be guaranteed to find the optimal answer!

Admissibility

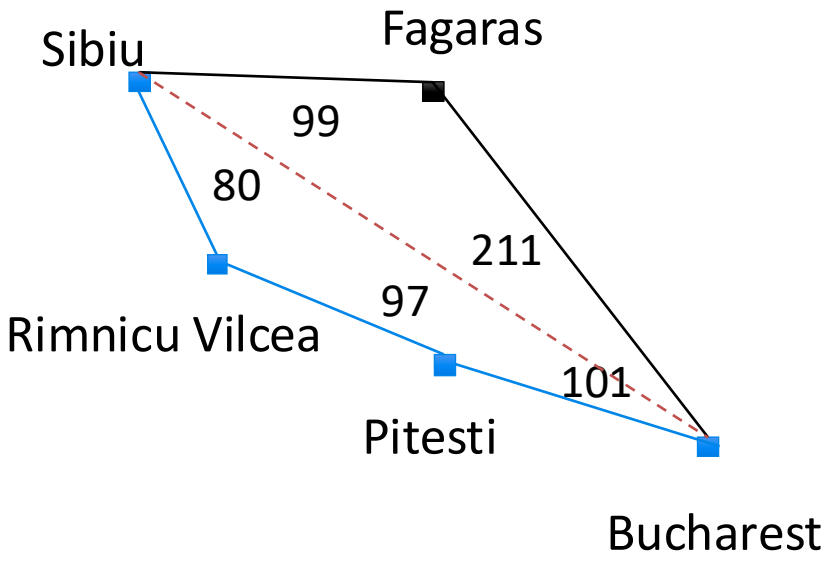
- What does it mean to be admissible?
  - The heuristic never overestimates the cost to reach the goal
  - $h(n)$ <= minimum achievable cost from n to goal

- Is the straight-line distance admissible?
  - Why or why not?

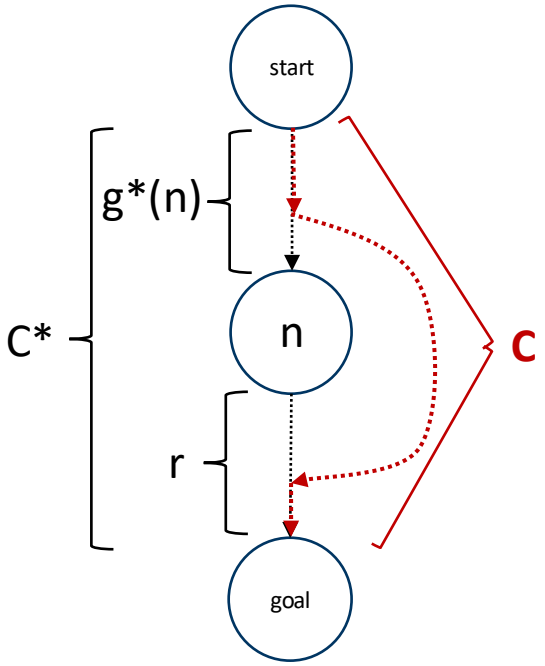
Admissibility of Straight-Line Distance

- Why is the straight-line distance admissible?
  - Straight line distance is the shortest possible path between any two points
  - It will never overestimate the actual path!

Legend:
- - Heuristic distance
— Shortest road path



A* with an admissible heuristic is cost-optimal



Proof by contradiction: Suppose the optimal path cost is C*,
but the algorithm finds a worse path with cost **C** > C* even though h is admissible.

If all nodes on the optimal path were expanded,
we would have returned the optimal solution with lower cost C*.
So there must be at least one node `n` on the optimal path that was not expanded.

Let g*(n) be the cost from start → n on the optimal path.
Let r be the remaining cost from n → goal on the optimal path. (Actual, not estimated.)

Since `n` on optimal path wasn't expanded, `f(n)= g*(n) + h(n)` must have been too high to be selected. Specifically: `f(n) ≥ C > C*`

$$
\begin{aligned}
f(n) &= g^*(n) + h(n) \\
&\le g^*(n) + r \\
&= C^*
\end{aligned}
$$

$h(n) \le r$ by admissibility, so…

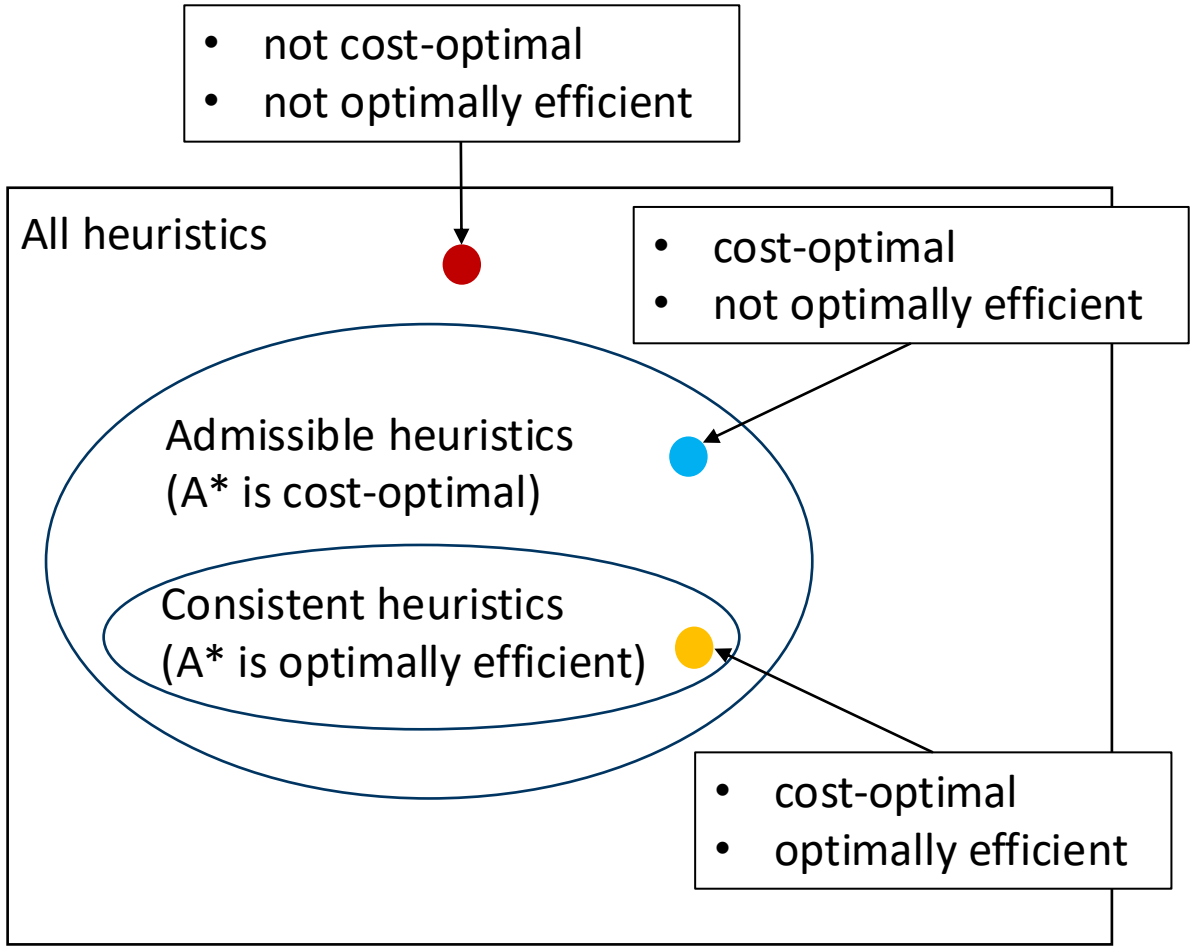
$C^* = g^*(n) + r$, so…

contradiction!

Thus: $f(n) \le C^*$.

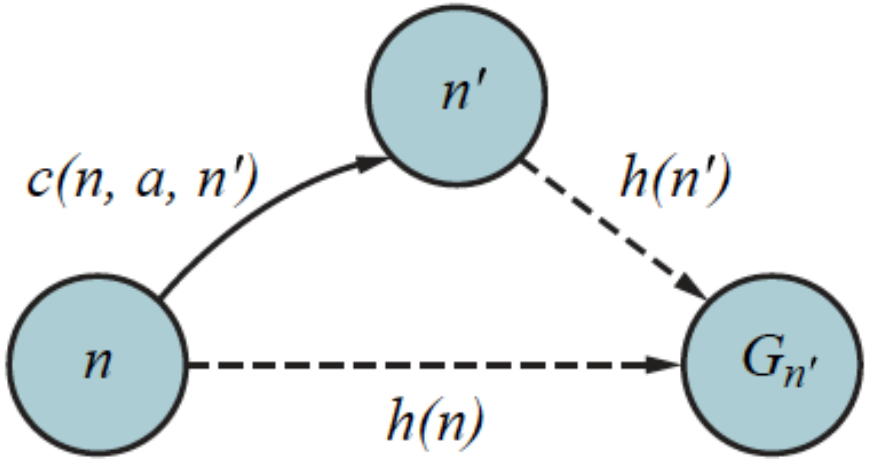
So our supposition was incorrect, and A* must be optimal.

Reminder: Heuristic Characteristics



Optimally efficient:
No algorithm can expand fewer nodes and still be guaranteed to find the optimal answer!

Consistency (Monotonicity)



- Stronger condition than admissibility
  - Every consistent heuristic is admissible
  - (Not vice versa)

- A heuristic function is consistent if:
  - For every node *n* and for every successor *n'* of *n* generated by action *a:*

$$h(n) \leq c(n, a, n') + h(n')$$

Consistency (Monotonicity)

- "h(n) is consistent" means:
  - For every node *n*,
    and for every successor *n'* of *n* generated by action *a*,
    it must be that $\boldsymbol{h(n)} \leq \boldsymbol{c(n,a,n')} + \boldsymbol{h(n')}$

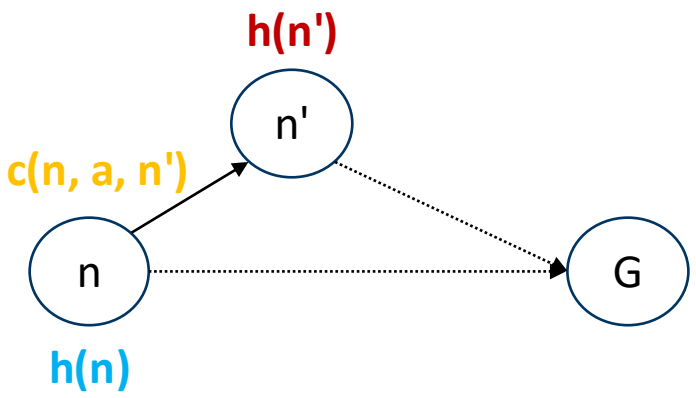
Some informal intuition:

- Suppose h(n) is admissible (optimistic or exact).

- h(n)
  is an estimate of the cost from n to G
  - There's the possibility of a lot of optimism in this estimate

- c(n,a,n') + h(n')
  is an estimate of the cost from n to G on a path starting with n→n'
  - Since we've specified exactly some of the path cost (c(n,a,n')), there's less room to be optimistic in what's left (from n').
  - So we'd expect $\mathrm{h(n)} \leq \mathrm{c(n,a,n')} + \mathrm{h(n')}$

more opt. less or eq. opt.



Intuitively:
As we go down the path towards a goal,

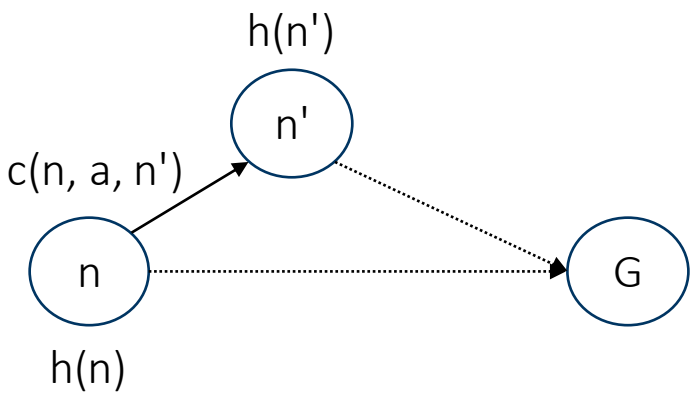
f(n) is non-decreasing,

because the estimated optimistic part (h)

is gradually replaced by more exact parts (g).

Consistency (Monotonicity)

- "h(n) is consistent" means:
  - For every node n,
    and for every successor n' of n generated by action a,
    it must be that $\mathbf{h(n)} \leq \mathbf{c(n, a, n')} + \mathbf{h(n')}$



- Example: If our path is A→B→C→G and h is consistent, then:

$$
\begin{aligned}
f(A) &= g(A) + h(A) \\
&= h(A) \\
&\leq c(A,B) + h(B) && = f(B) \\
&\leq c(A,B) + c(B,C) + h(C) && = f(C) \\
&\leq c(A,B) + c(B,C) + c(C,G) + h(G) && = f(G) \\
&= c(A,B) + c(B,C) + c(C,G)
\end{aligned}
$$

Thus, we have:

$$f(A) \leq f(B) \leq f(C) \leq f(G)$$

Intuitively:
As we go down the path towards a goal,

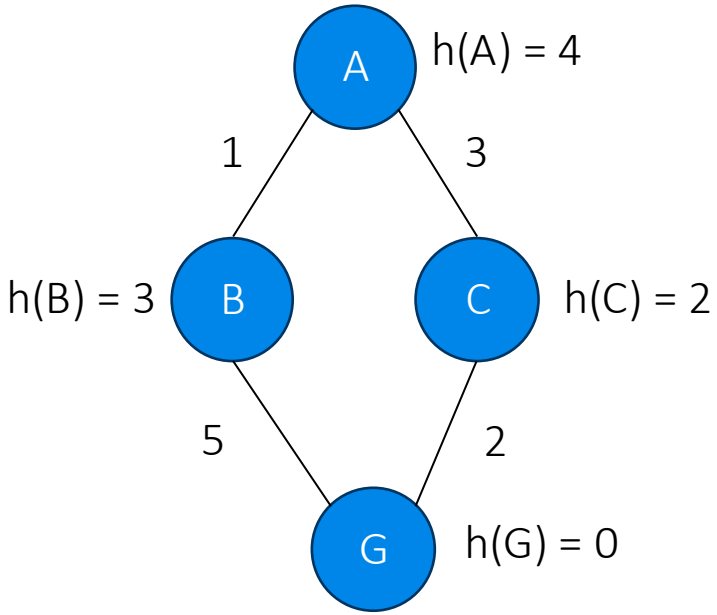
f(n) is non-decreasing,

because the estimated optimistic part (h)

is gradually replaced by more exact parts (g).

If we use a consistent heuristic, then:
the first time a node containing state s is chosen for expansion, it will be the node that represents the optimal path cost to s.

proof omitted…

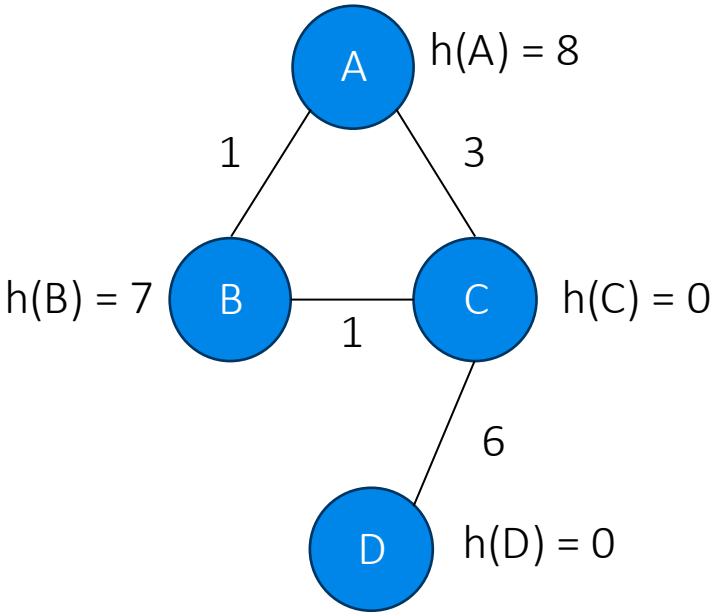


Note: The heuristic in the figure is consistent.

How will A* behave? (Starting at A, going to G.)

A: 0 + 4 = 4	Expansion #1 – this is the best path to A
AB: 1 + 3 = 4	Expansion #2 – this is the best path to B
AC: 3 + 2 = 5	Expansion #3 – this is the best path to C
ABG: 6 + 0 = 6	
ACG: 5 + 0 = 5	Expansion #4 – this is the best path to G

If we use an inconsistent heuristic, then:
the first time a node containing state s is chosen for expansion,
it will not necessarily be the node
that represents the optimal path cost to s.
Furthermore, A* may expand more nodes than it would with a consistent heuristic while searching for a solution.



The heuristic in the figure is inconsistent:
h(C) is bad…
(A,C): 8 ≤ 3 + 0 ? No!
(B,C): 7 ≤ 1 + 0 ? No!

This ex. shows the "expand more nodes" issue.
For non-cost-optimal, we'd need an inadmissible h.

How will A* behave? (Starting at A, going to D.)

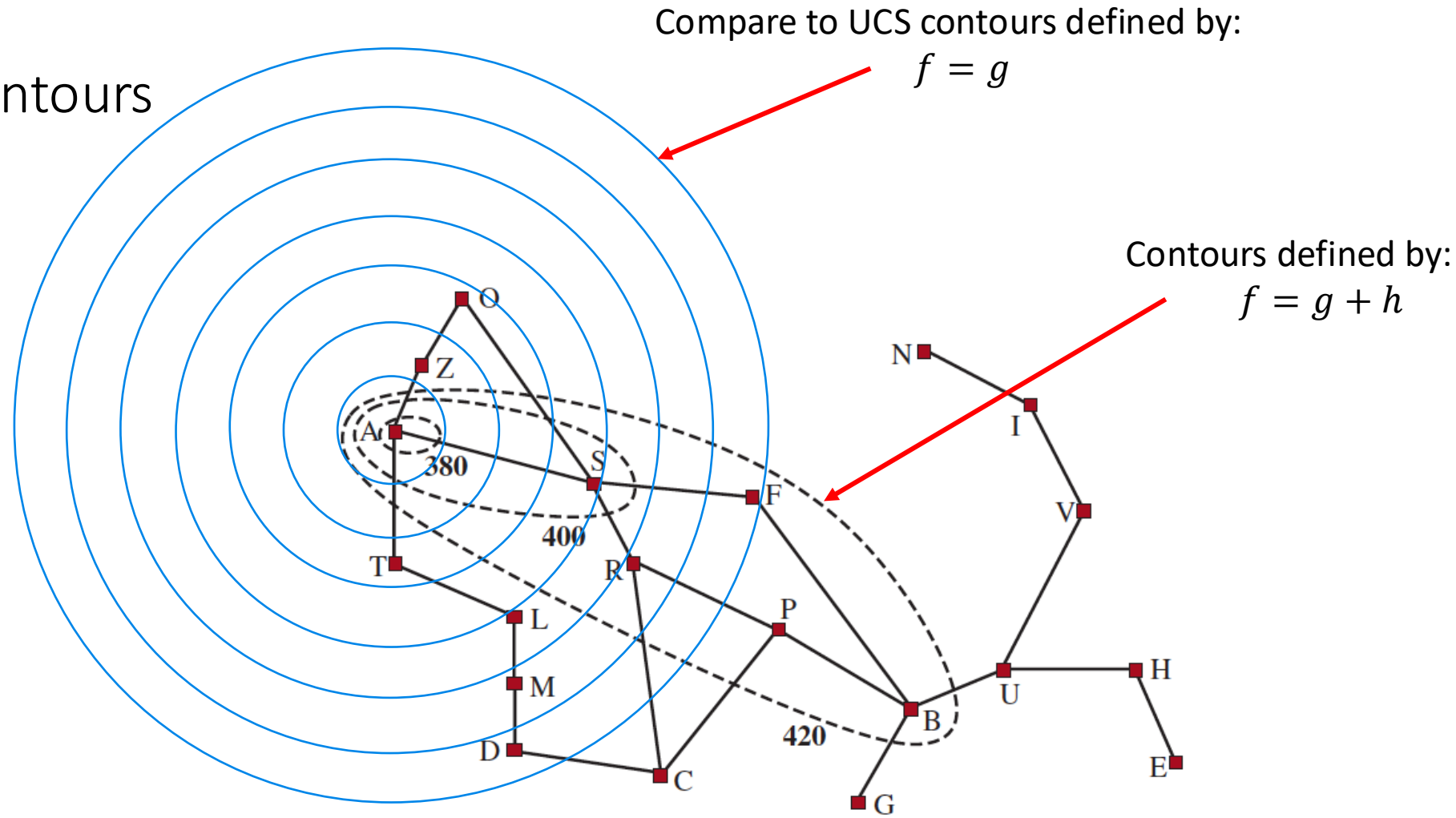
A: 0 + 8 = 8	Expansion #1
AB: 1 + 7 = 8	Expansion #3
AC: 3 + 0 = 3	Expansion #2 – first time expanding C
ACD: 9 + 0 = 9	
ACB: 4 + 7 = 11	
ABC: 2 + 0 = 2	Expansion #4 – second time expanding C, with better path to C
ABCD: 8 + 0 = 8	Expansion #5

Extra expansion compared to what would have happened with a consistent heuristic.

But inconsistency is rarely a big problem in practice, says: Felner, Ariel, et al. "Inconsistent heuristics in theory and practice." Artificial Intelligence 175.9-10 (2011): 1570-1603.

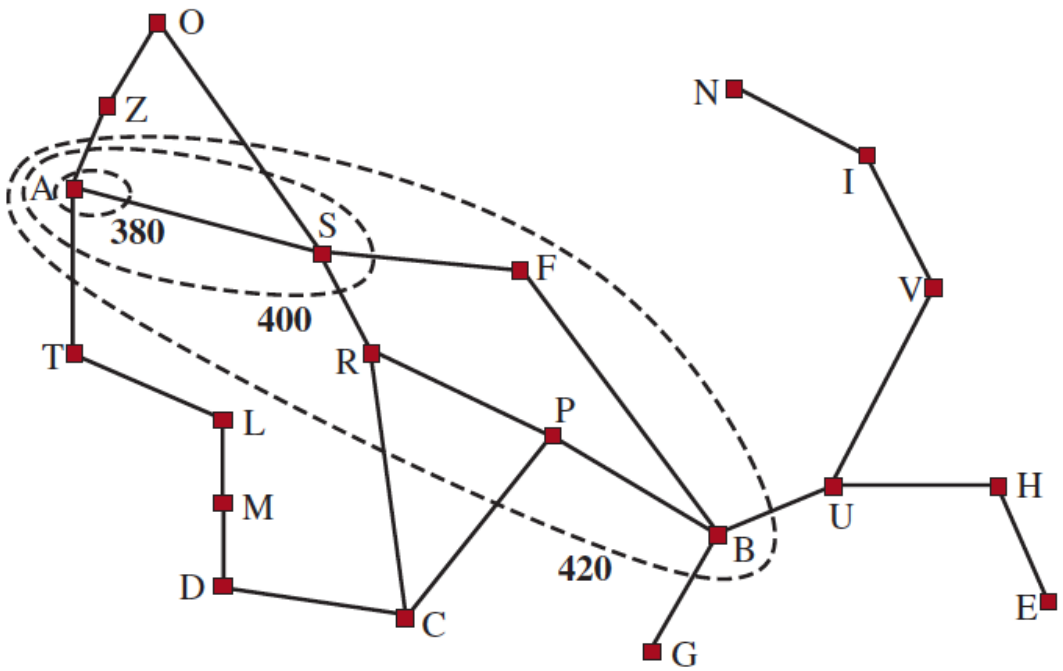
A* Search Path

- Contours



A* Search Path

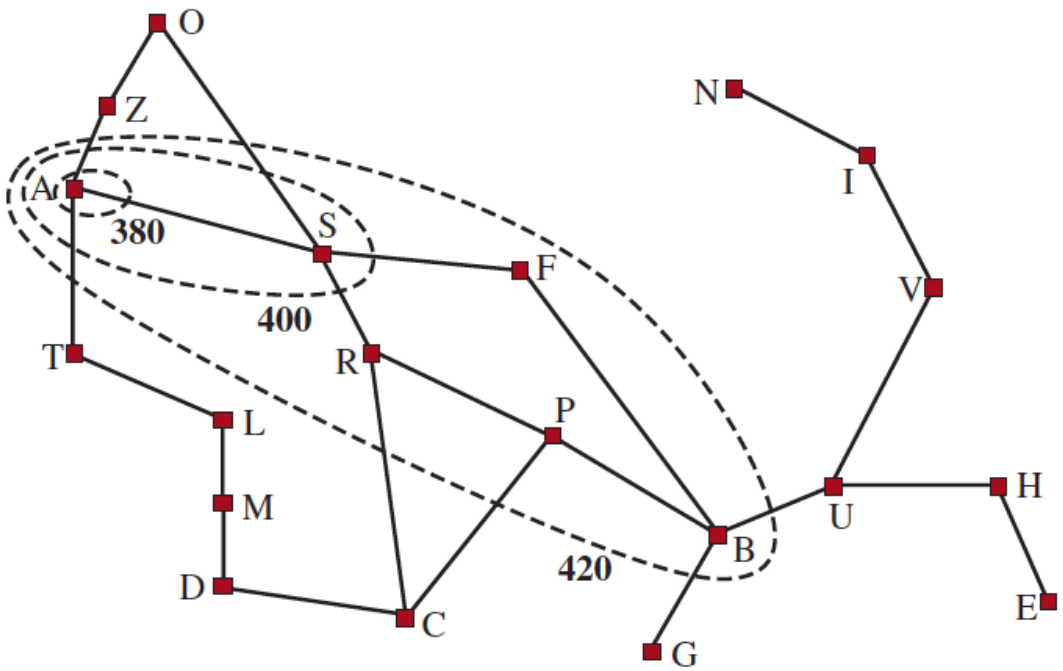
- Contours
  - Issue: Most problems have a number of nodes in the contour space exponential with the length of the solution!



A* Search Path

Possible solution: satisficing (Section 3.5.4)

- Contours
  - Issue: Most problems have a number of nodes in the contour space exponential with the length of the solution!



Completeness

- Requires only finitely many nodes with cost less than or equal to C*
  - True when step cost > small constant ϵ
  - True when b is finite

Limitations of A*

- Still has memory challenges!

- Are there solutions to get around the space problems without sacrificing optimality/completeness?
  - Yes! See Section 3.5.5 in the text
    - E.g., Iterative Deepening A* (IDA*)
    - E.g., when memory fills, drop the worst leaf
  - Downside: increase in computation time

What Should We Take Away?

- We must be able to construct appropriate heuristics!!!

Constructing Heuristics

The 8-Puzzle

- Rule: A tile adjacent to the blank space can slide into that space
- Goal: match a given configuration

Start State

7	2	4
5		6
8	3	1

Goal State

	1	2
3	4	5
6	7	8

How can we generate heuristics?

1. Just think of something (hard)
   - Can use domain knowledge
2. Relax the problem constraints
   - Solve a sub-problem
3. Learn the heuristics (database)

Start State

7	2	4
5		6
8	3	1

Goal State

	1	2
3	4	5
6	7	8

Relax the problem constraints

- Generate Heuristics by Relaxation
  - Calculate the exact distance for the relaxed version of the problem (you can break the rules of the environment)

- Two (of many) possibilities:
  - Allow tile to be teleported
    h_1 = # of misplaced tiles
    (e.g., 8)
  - Allow move to adjacent square even if square is occupied
    h_2 = Manhattan Distance
    (e.g., 3+1+2+2+2+3+3+2 = 18)

    This is better than h_1 – we'll see why.

Start State

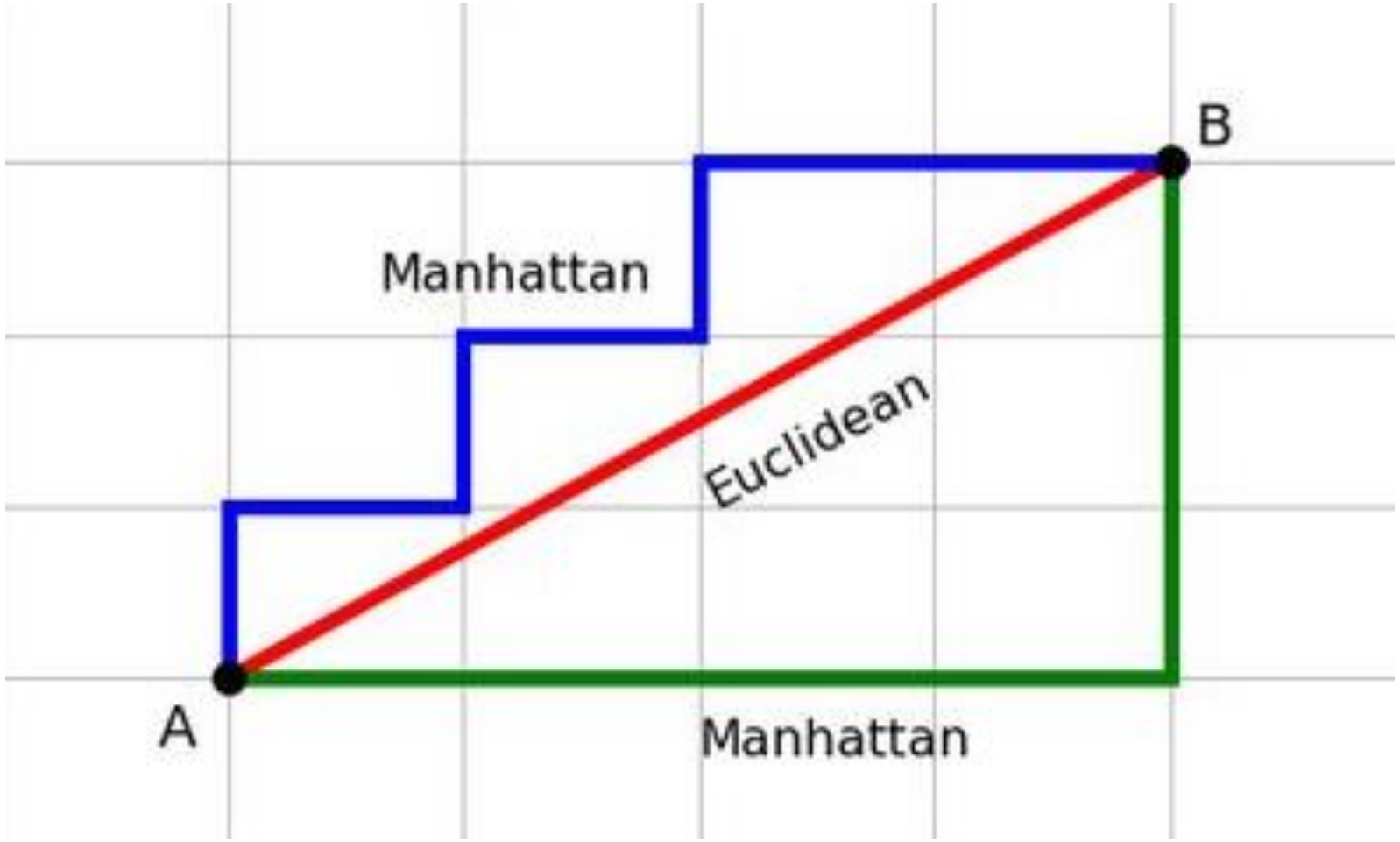
7	2	4
5		6
8	3	1

Goal State

	1	2
3	4	5
6	7	8

Image from https://www.omnicalculator.com/math/manhattan-distance

Manhattan Distance



Quick Exercise

- Are the two heuristics on the previous slide admissible?

Comparing Heuristics

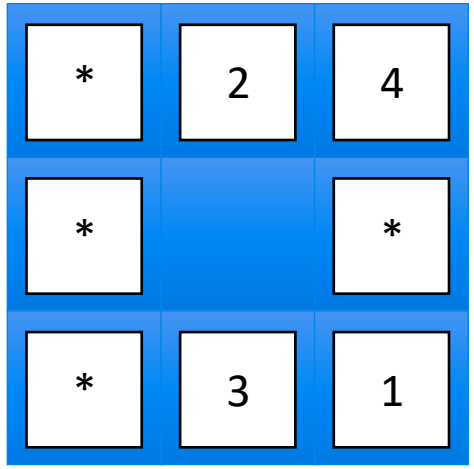
- Consider the following heuristics:
  - h_1 = # of misplaced tiles
  - h_2 = Manhattan Distance
- Which heuristic is better?
  - If any of them is consistent, then that one
  - Next best: If both are admissible, the one that will expand fewer nodes
- We know that A* will expand all nodes with f(n) <= C*
  - h_1 and h_2 have the same path cost, *g(n)*
  - The difference is in the heuristic…

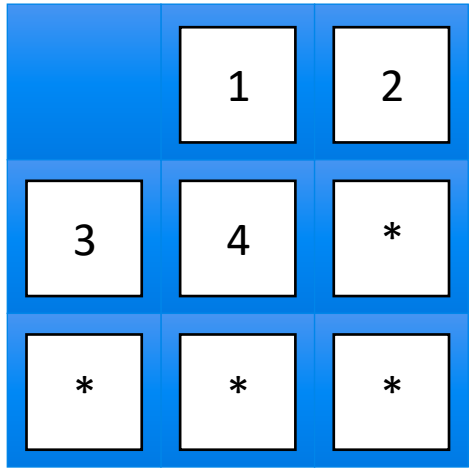
Heuristics – Choosing the Best

- How to choose?
- Don't choose!
  - $h(n) = \max\{h_1(n), \ldots, h_m(n)\}$
- Why is this okay?

Solve a sub-problem

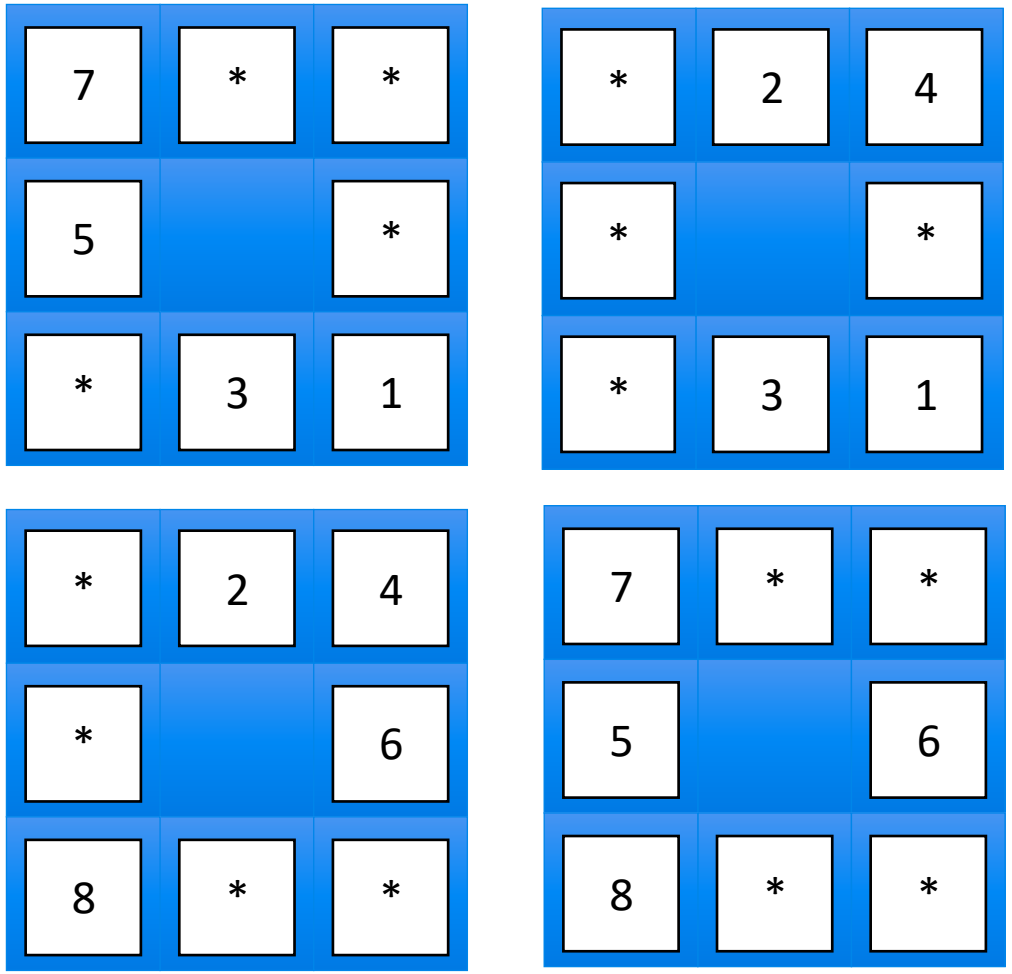
- Solve subproblems instead of the entire problem
  - Subproblems must be easy to solve!
- Example:
  - Get tiles 1,2,3,4 into their proper positions
- The result is a lower bound on the full problem
  - Can sometimes be more accurate than Manhattan distance!





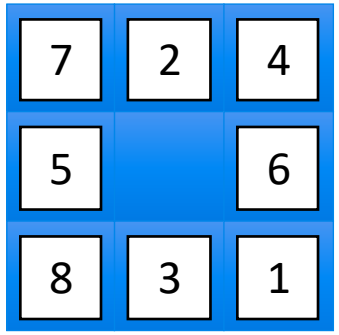
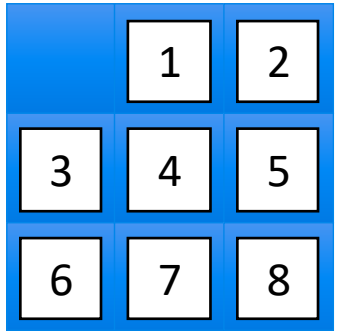
Heuristics – Pattern Database

- Create a database with solutions to all possible subproblems
- Compute admissible heuristic for each complete state by looking up in database
- Combine heuristics by taking the maximum



Heuristics – Machine Learning

- Simulate or ask users to play the game many times
- Curate a database of board positions and numbers of steps to given solutions
- Train an algorithm to predict how close a given board position is to the identified goal state



…